



The Open University

Mathematics and Computing: Level 2
M256 Software development with Java

M256

Handbook

Contents

1	Software development in M256	3
2	M256 approach to designing a coordinating method for a use case	4
3	Hospital System requirements document	5
4	Hospital System initial structural model	7
5	Hospital System implementation model	9
	Structural model	9
	Dynamic models	16
6	Shopping System requirements document	27
7	Shopping System initial structural model	31
8	DVD Library System requirements document	33
9	DVD Library System initial structural model	34

1

Software development in M256

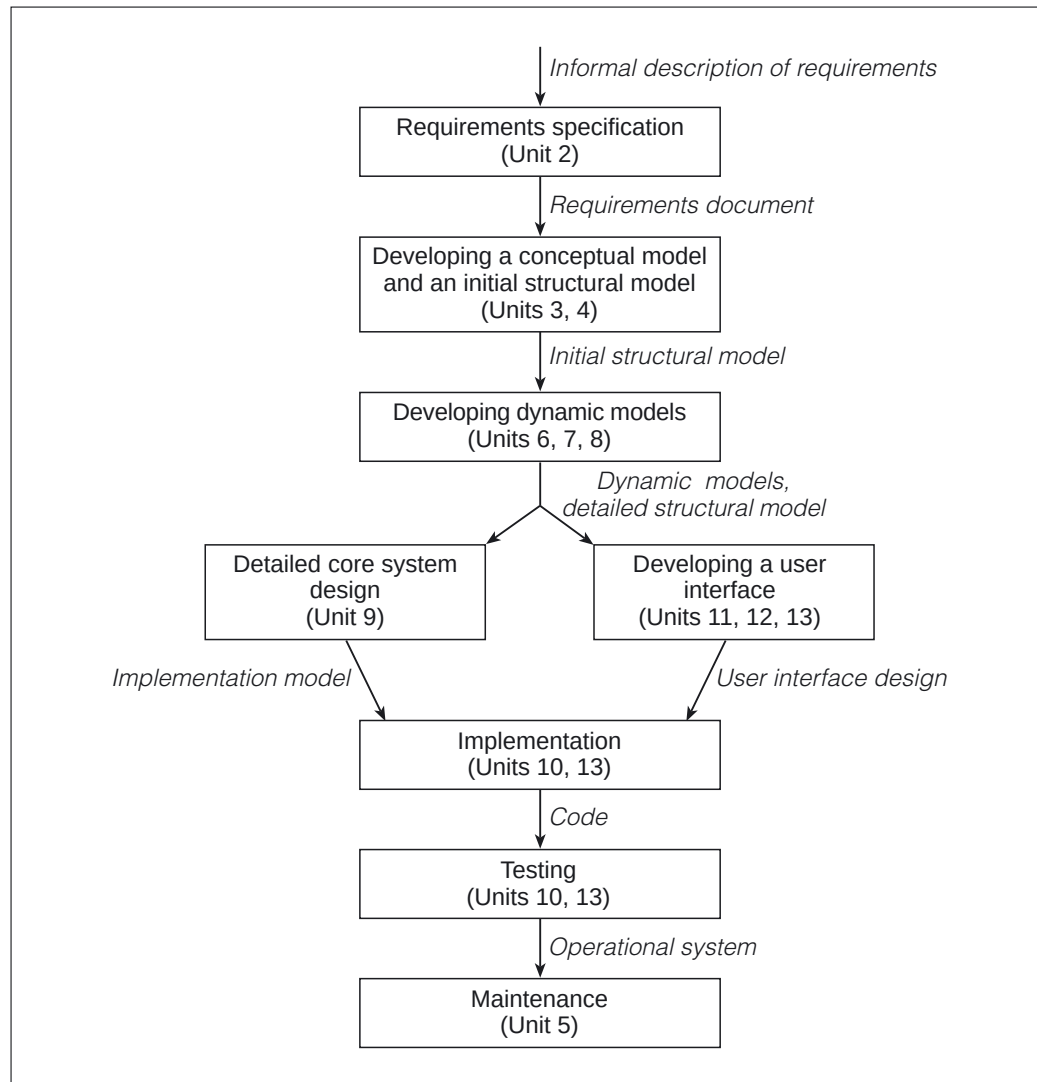


Figure 1 Software development in M256.

2

M256 approach to designing a coordinating method for a use case

- 1 Create an object diagram illustrating the example objects of a typical scenario.
- 2 Construct a sequence diagram illustrating the communication between the user interface and the coordinating object.
- 3 Specify the coordinating method.
- 4 Set out a walk-through for the use case – the steps required, for the typical scenario, to achieve the coordinating method specification – and extend the sequence diagram to show how the steps of the walk-through can be achieved by sending messages.
- 5 From the walk-through and sequence diagram:
 - ▶ identify method specifications and responsibilities for holding link references, and add them to the relevant class descriptions;
 - ▶ add (if not already present) a navigation arrow to the class diagram which corresponds to each class's responsibility for holding a link reference.

3

Hospital System requirements document

The requirements document for the Hospital System

System domain

In the hospital, there are a number of wards, each of which may be empty or have on it one or more patients. There are two types of ward, male wards and female wards. A ward can only have patients of the specified sex on it. Every ward has a fixed capacity, which is the maximum number of patients that can be on it at one time (i.e. the capacity is the number of beds in the ward). Different wards may have different capacities. Each ward has a unique name. The hospital has an Administration department that is responsible for recording information about the hospital's wards and the patients that are on each ward.

The doctors in the hospital are organised into teams, each of which has a unique team code (such as Orthopaedics A, or Paediatrics). Each team is headed by a consultant doctor who is the only consultant doctor in the team; the rest of the team are all junior doctors, at least one of whom must be at grade 1. Each doctor is in exactly one team. The Administration department keeps a record of these teams and the doctors allocated to each team.

Each patient is on a single ward and is under the care of a single team of doctors; the consultant who heads that team is responsible for the patient. A patient may be treated by any number of doctors but they must all be in the team that cares for the patient. A doctor can treat any number of patients.

Use cases

The Hospital system will provide support for the administration of the hospital by allowing hospital administrators to do the following.

A Admit Patient. The administrator provides the system with the patient's name, sex and date of birth and identifies the team that cares for the patient.

The system identifies a ward of the appropriate type with empty beds; if there is a choice of such wards one of those with the largest number of empty beds is chosen.

If there is no appropriate ward with empty beds the administrator is informed of this fact.

If there is an appropriate ward with empty beds the system does the following two things.

- 1 It records:
 - ▶ the patient's name, sex and date of birth;
 - ▶ that the patient is under the care of the given team;
 - ▶ that the consultant that heads the team is responsible for the patient;
 - ▶ that the patient is on the ward.
- 2 It informs the administrator of the ward to which the patient has been admitted.

- B Treat Patient.** The administrator identifies the patient and the doctor; the doctor must be in the team that cares for the patient. The system records that the doctor has treated the patient.
- C Discharge Patient.** The administrator identifies the discharged patient. All information relating to that patient is removed from the system.
- D Transfer Patient.** The administrator identifies the patient and the new ward. The new ward should be of the appropriate type and have at least one free bed. The system records:
 - ▶ the removal of the patient from their current ward;
 - ▶ the admission of the patient to the new ward.
- E List Ward's Patients.** The administrator identifies the ward. For each patient on the ward the system displays the patient's name and age in years.
- F List Team's Patients.** The administrator identifies the team. For each patient cared for by the team the system displays:
 - ▶ the patient's name;
 - ▶ the name of the ward that the patient is on.
- G List Patient's Treatment.** The administrator identifies the patient. The system displays:
 - ▶ the name of the consultant responsible for the patient;
 - ▶ the team code of the team that cares for the patient;
 - ▶ the name of each doctor who has treated the patient and, for each such junior doctor, their grade (1, 2 or 3).

Acceptance tests

- A** Test that, when there is an appropriate ward available:
 - ▶ the admission of a patient is recorded correctly;
 - ▶ the administrator is informed of the ward to which the patient has been allocated.

Test that, when no appropriate ward is available, an attempt to record the admission of a patient results in the administrator being informed that there is no appropriate ward available.
- B** Test that the treatment of a patient, by a doctor on the team that cares for the patient, is recorded correctly.
- C** Test that the discharge of a patient is recorded correctly.
- D** Test that the transfer of a patient to an appropriate ward with an empty bed is recorded correctly.
- E** Test that a list of patients on a ward is produced correctly when there are no patients on the ward.

Test that a list of patients on a ward is produced correctly when there are some patients on the ward.
- F** Test that a list of patients cared for by a team is produced correctly when there are no patients cared for by the team.

Test that a list of patients cared for by a team is produced correctly when there are some patients cared for by the team.
- G** Test that a list of a patient's treatment is produced correctly when no doctors have treated the patient.

Test that a list of a patient's treatment is produced correctly when some doctors have treated the patient.

4

Hospital System initial structural model

Class diagram

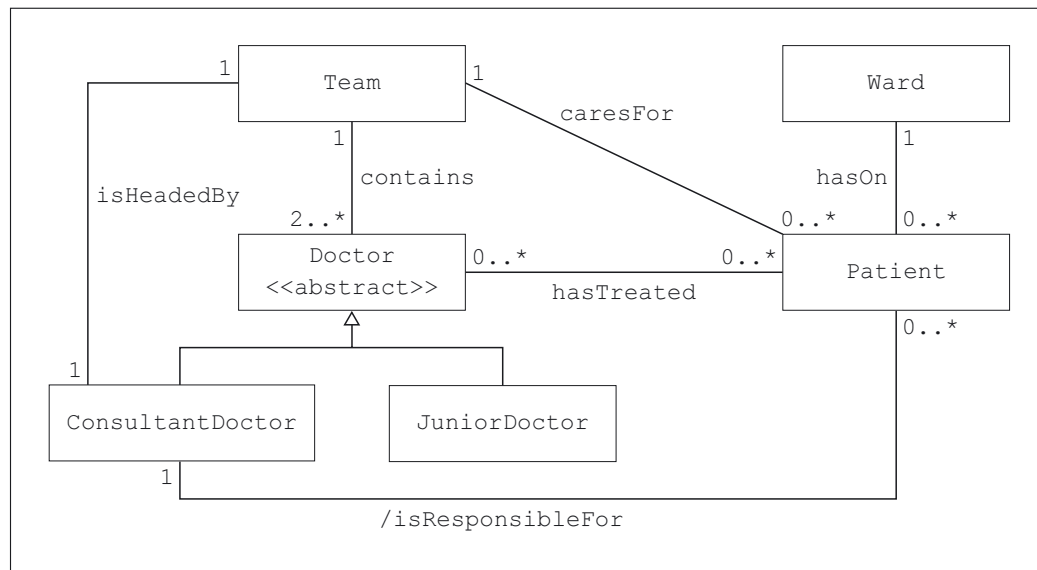


Figure 2 Class diagram for the Hospital System initial structural model

Class descriptions

Class Ward A hospital ward

Attributes

name	The unique name of the ward
type	Whether the ward is for male or female (M or F) patients
capacity	The maximum number of patients that can be on the ward at any one time
/numberOfFreeBeds	The number of free beds on the ward

Class Patient A patient in the hospital

Attributes

name	The name of the patient
/sex	The sex (M or F) of the patient
dateOfBirth	The date of birth of the patient
/age	The age of the patient in years

Class	Team	A team of doctors
	Attributes	
	code	The unique code of the team
Class	Doctor	A doctor at the hospital Abstract: generalises JuniorDoctor and ConsultantDoctor
	Attributes	
	name	The name of the doctor
Class	JuniorDoctor	A junior doctor at the hospital Specialises Doctor
	Attributes	
	grade	The grade (1, 2 or 3) of the junior doctor
Class	ConsultantDoctor	A consultant doctor at the hospital Specialises Doctor
	Attributes	
	None	

Invariants

- 1 The sex attribute of any Patient object has the same value as the type attribute of the linked Ward object.
- 2 For each Patient object, the value of its age attribute is equal to the difference (in years) between the current date and the value of its dateOfBirth attribute.
- 3 For each Ward object, the value of its numberOfFreeBeds attribute is equal to the value of its capacity attribute minus the number of Patient objects to which it is linked.
- 4 Each Team object is linked to at least one JuniorDoctor object for which the grade attribute has a value of 1.
- 5 The ConsultantDoctor object linked to a Team object via isHeadedBy is linked to the same Team object via contains.
- 6 The ConsultantDoctor object linked to any Patient object via isResponsibleFor is linked via isHeadedBy to the Team object to which that Patient object is linked.
- 7 If aPatient is any Patient object, then any Doctor object linked to aPatient is also linked to the Team object which is linked to aPatient.

5

Hospital System implementation model

(The use cases *Discharge Patient* and *Transfer Patient* are omitted as they are not considered in the course materials.)

This model consists of:

- ▶ a structural model (class diagram, class descriptions and invariants);
- ▶ dynamic models (walk-throughs and sequence diagrams for particular scenarios corresponding to each use case).

Structural model

Class diagram

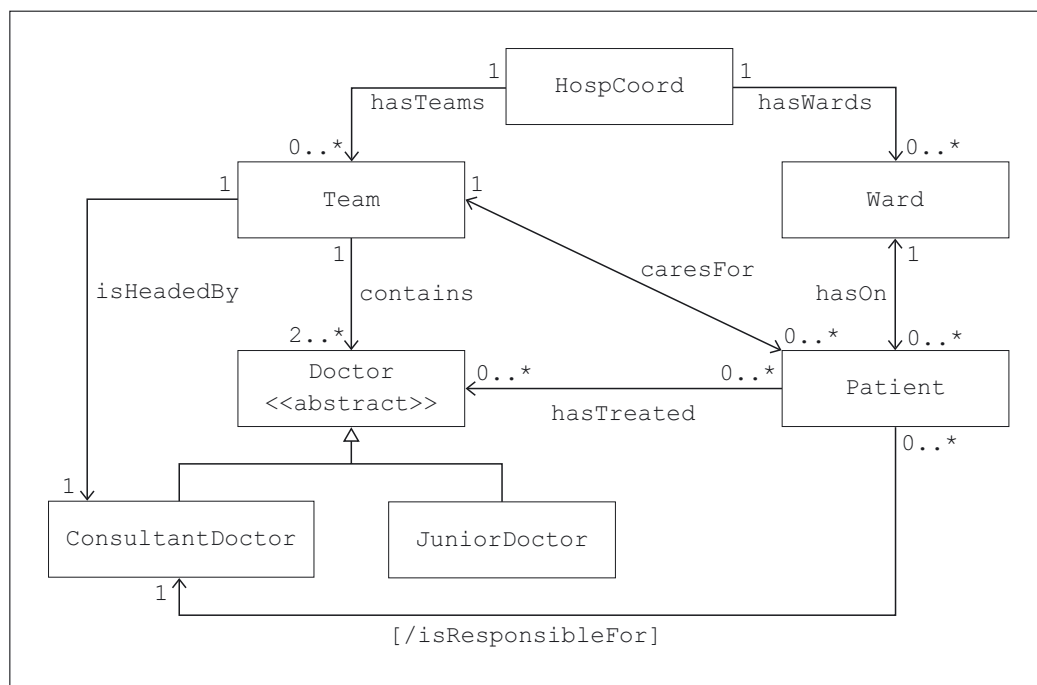


Figure 3 Class diagram for the Hospital System implementation model

Class descriptions

Package hospitalcore

Class HospCoord The coordinating class

Attributes

None

Links

private Collection<Ward> wards
References a collection of all Ward objects.

private Collection<Team> teams
References a collection of all Team objects.

Protocol

```
public Collection<Patient> getPatients(Ward aWard)
```

Post-condition: returns a collection of all the `Patient` objects linked to `aWard`.

```
public Map<Patient, Ward> getPatientsAndWards(Team aTeam)
```

Post-condition: returns a map of pairs of `Patient` and `Ward` objects, such that for each `Patient` object `aPatient` linked to `aTeam`, the map contains the key-value pair (`aPatient`, `aWard`), where `aWard` is linked to `aPatient`.

```
public ConsultantDoctor getConsultantDoctor(Patient aPatient)
```

Post-condition: returns the `ConsultantDoctor` object linked to `aPatient`.

```
public Team getTeam(Patient aPatient)
```

Post-condition: returns the `Team` object linked to `aPatient`.

```
public Collection<Doctor> getDoctors(Patient aPatient)
```

Post-condition: returns a collection of all the `Doctor` objects linked to `aPatient`.

```
public void recordTreatment(Patient aPatient, Doctor aDoctor)
```

Pre-condition: `aDoctor` and `aPatient` must be linked to the same `Team` object; if not, an exception is thrown.

Post-condition: `aDoctor` is linked to `aPatient`.

```
public Ward admit(Name aName, Sex aSex, M256Date aDate, Team aTeam)
```

Post-condition: if there is no `Ward` object of the appropriate type with at least one free bed then `null` is returned.

Otherwise a new `Patient` object, `aPatient`, is created with the supplied attribute values, and age according to `aDate`, and:

- (i) `aPatient` is linked to `aWard`, where `aWard` is a `Ward` object of the appropriate type with the greatest number of free beds, and `numberOfFreeBeds` of `aWard` is decremented;
- (ii) `aPatient` is linked to `aTeam`;
- (iii) `aPatient` is linked to the `ConsultantDoctor` object that is linked to `aTeam`;
- (iv) `aWard` is returned.

```
public Collection<Ward> getWards()
```

Post-condition: returns a collection of all the `Ward` objects.

```
public Collection<Team> getTeams()
```

Post-condition: returns a collection of all the `Team` objects.

```
public Collection<Patient> getPatients()
```

Post-condition: returns a collection of all the `Patient` objects.

```
public Collection<Doctor> getDoctors(Team aTeam)
```

Post-condition: returns a collection of all the `Doctor` objects linked to `aTeam`.

```
public String toString()
```

Post-condition: returns a string representation of all `Team` and `Ward` objects.>

Class `Ward` A hospital ward

Attributes

`private String name` The unique name of the ward

`private Sex type` Whether the ward is for male or female (M or F) patients

private int capacity The maximum number of patients that can be on the ward at any one time

[int /numberOfFreeBeds The number of free beds on the ward]

Links

private Collection<Patient> patients

References a collection of all the linked Patient objects.

Protocol

public String getName()

Post-condition: returns name.

public Sex getType()

Post-condition: returns type.

public int getCapacity()

Post-condition: returns capacity.

public int getNumberOfFreeBeds()

Post-condition: returns numberOfFreeBeds.

public String toString()

Post-condition: returns a string representation of the receiver's name, type, capacity and linked Patient objects.

Collection<Patient> getPatients()

Post-condition: returns a collection of all the linked Patient objects.

void addPatient(Patient aPatient)

Post-condition: a reference to aPatient is recorded; numberOfFreeBeds is decremented.

Class Team A team of doctors

Attributes

private String code The unique code of the team

Links

private Collection<Patient> patients

References a collection of all the linked Patient objects.

private ConsultantDoctor consultantDoctor

References the linked ConsultantDoctor object.

private Collection<Doctor> doctors

References a collection of all the linked Doctor objects.

Protocol

public String getCode()

Post-condition: returns code.

public String toString()

Post-condition: returns a string representation of the receiver's code, linked Doctor objects and linked Patient objects.

Map<Patient, Ward> getPatientsAndWards()

Post-condition: returns a map of pairs of Patient and Ward objects, such that for each Patient object, aPatient, linked to the receiver, the map contains the key-value pair (aPatient, aWard) where aWard is linked to aPatient.

```
void addPatient(Patient aPatient)
    Post-condition: a reference to aPatient is recorded.
ConsultantDoctor getConsultantDoctor()
    Post-condition: returns the linked ConsultantDoctor object.
boolean contains(Doctor aDoctor)
    Post-condition: returns true if aDoctor is linked to the receiver, false
    otherwise.
Collection<Doctor> getDoctors()
    Post-condition: returns a collection of all the linked Doctor objects.
```

Class Patient A patient in the hospital

Attributes

private Person person The person who is a patient

Links

private Ward ward
References the linked Ward object.

private Team team
References the linked Team object.

private Collection<Doctor> doctors
References a collection of all the linked Doctor objects.

[ConsultantDoctor consultantDoctor
References the linked ConsultantDoctor object.]

Constructor

Patient(Name aName, Sex aSex, M256Date aDate)
Post-condition: initialises a new Patient object with the given attribute values and age according to aDate.

Protocol

public Name getName()
Post-condition: returns name.

public Sex getSex()
Post-condition: returns sex.

public int getAge()
Post-condition: returns age.

public M256Date getDateOfBirth()
Post-condition: returns dateOfBirth.

public String toString()
Post-condition: returns a string representation of name, sex and dateOfBirth.

Ward getWard()
Post-condition: returns the linked Ward object.

Team getTeam()
Post-condition: returns the linked Team object.

Collection<Doctor> getDoctors()
Post-condition: returns a collection of all the linked Doctor objects.

```
ConsultantDoctor getConsultantDoctor()
```

Post-condition: returns the linked ConsultantDoctor object.

```
void recordTreatmentBy(Doctor aDoctor)
```

Pre-condition: aDoctor and the receiver must be linked to the same Team object; if not, an exception is thrown.

Post-condition: a reference to aDoctor is recorded.

```
void admit(Ward aWard, Team aTeam)
```

Post-condition: A reference to aWard is recorded, aWard records a reference to the receiver, and numberOfFreeBeds of aWard is decremented. A reference to aTeam is recorded, and aTeam records a reference to the receiver. A reference to the ConsultantDoctor object linked to aTeam is recorded.

Class Doctor A doctor at the hospital
 Abstract: generalises JuniorDoctor and
 ConsultantDoctor

Attributes

private Person person The person who is a doctor

Links

None

Protocol

```
public Name getName()
```

Post-condition: returns name.

```
public String toString()
```

Post-condition: returns a string representation of name.

Class JuniorDoctor A junior doctor at the hospital
 Specialises Doctor

Attributes

private Grade grade The grade (1, 2 or 3) of the junior doctor

Links

None

Protocol

```
public Grade getGrade()
```

Post-condition: returns grade.

```
public String toString()
```

Post-condition: returns a string representation of name and grade.

Class ConsultantDoctor A consultant doctor at the hospital
 Specialises Doctor

Attributes

None

Links

None

Protocol

None

Package m256people**Class** Person A person**Attributes**

private Name name	The name of the person
private Sex sex	The sex (M or F) of the person
private M256Date dateOfBirth	The date of birth of the person
[int /age	The age of the person in years]

Constructors

```
public Person(Name aName, Sex aSex, M256Date aDate)
    Post-condition: initialises a new Person object with the given attribute
    values and age according to aDate.

public Person(Name aName)
    Post-condition: initialises a new Person object with the given name value.
    Sets sex, dateOfBirth and age to null.

public Person(Person aPerson)
    Post-condition: initialises a new Person object with the same attribute
    values as aPerson.
```

Protocol

```
public Name getName()
    Post-condition: returns name.

public Sex getSex()
    Post-condition: returns sex.

public int getAge()
    Post-condition: returns age.

public M256Date getDateOfBirth()
    Post-condition: returns dateOfBirth.

public Name setName(Name aName)
    Post-condition: sets name to aName.

public String toString()
    Post-condition: returns a string representation of the receiver's name, sex
    and dateOfBirth.
```

Class Name A person's name
This class is immutable. It implements Comparable<Name>**Attributes**

private String title	The title of the person
private String firstName	The first name of the person
private String surname	The surname of the person

Constructor

```
public Name(String aTitle, String aFirstName, String aSurname)
```

Post-condition: initialises a new Name object with the given attribute values.

Protocol

```
public String getTitle()
```

Post-condition: returns title.

```
public String getFirstName()
```

Post-condition: returns firstName.

```
public String getSurname()
```

Post-condition: returns surname.

```
public boolean equals(Object o)
```

Post-condition: returns true if o is a Name object with title, firstName and surname equal to those of the receiver, otherwise returns false.

```
public int hashCode()
```

Post-condition: returns the hashCode of the receiver. Consistent with equals().

```
public int compareTo(Name aName)
```

Post-condition: returns a negative integer if the receiver is alphabetically before aName, a positive integer if the receiver is alphabetically after aName and zero otherwise. Name objects are compared using surname, firstName and title in that order.

```
public String toString()
```

Post-condition: returns a string representation of the receiver's title, firstName and surname.

Enum Sex The sex of a person

Values: M and F

Protocol

```
public String toString()
```

Post-condition: returns a string representation of the sex.

Enum Grade The grade of a junior doctor

Values: ONE, TWO and THREE

Protocol

```
public String toString()
```

Post-condition: returns a string representation of the grade.

Invariants

- 1 The sex attribute of any Patient object has the same value as the type attribute of the linked Ward object.
- 2 For each Patient object, the value of its age attribute is equal to the difference (in years) between the current date and the value of its dateOfBirth attribute.
- 3 For each Ward object, the value of its numberOfFreeBeds attribute is equal to the value of its capacity attribute minus the number of Patient objects to which it is linked.
- 4 Each Team object is linked to at least one JuniorDoctor object for which the grade attribute has a value of 1.

- 5 The ConsultantDoctor object linked to a Team object via isHeadedBy is linked to the same Team object via contains.
- 6 The ConsultantDoctor object linked to any Patient object via isResponsibleFor is linked via isHeadedBy to the Team object to which that Patient object is linked.
- 7 If aPatient is any Patient object, then any Doctor object linked to aPatient is also linked to the Team object which is linked to aPatient.

Dynamic models

A: Admit Patient

Coordinating method:

```
Ward admit(Name aName, Sex aSex, M256Date aDate, Team aTeam)
```

Object diagrams for a typical scenario (in which there is a suitable ward):

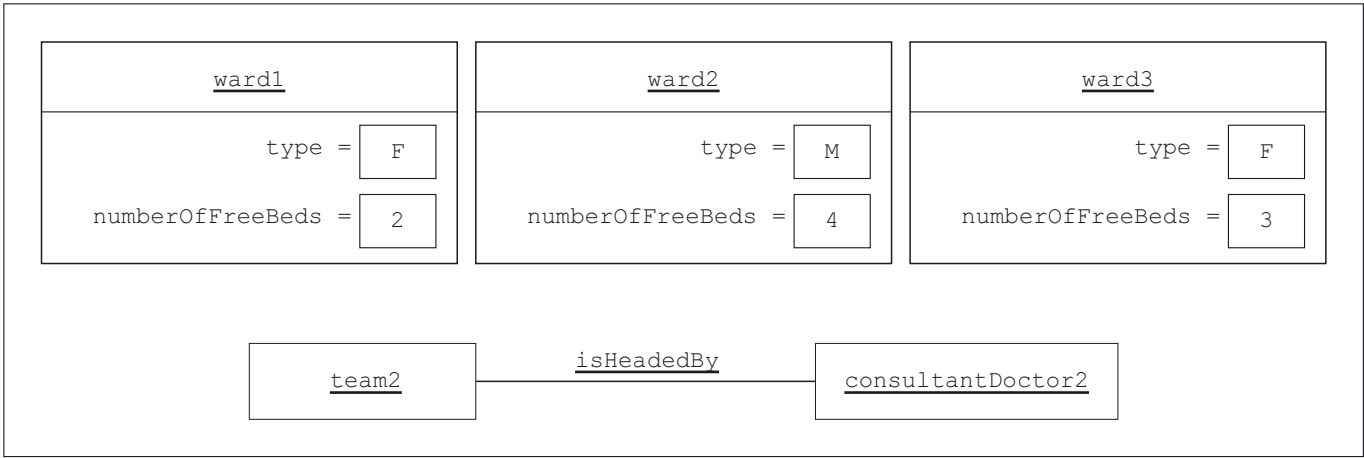


Figure 4 Object diagram for the scenario before *Admit Patient* (a suitable ward exists)

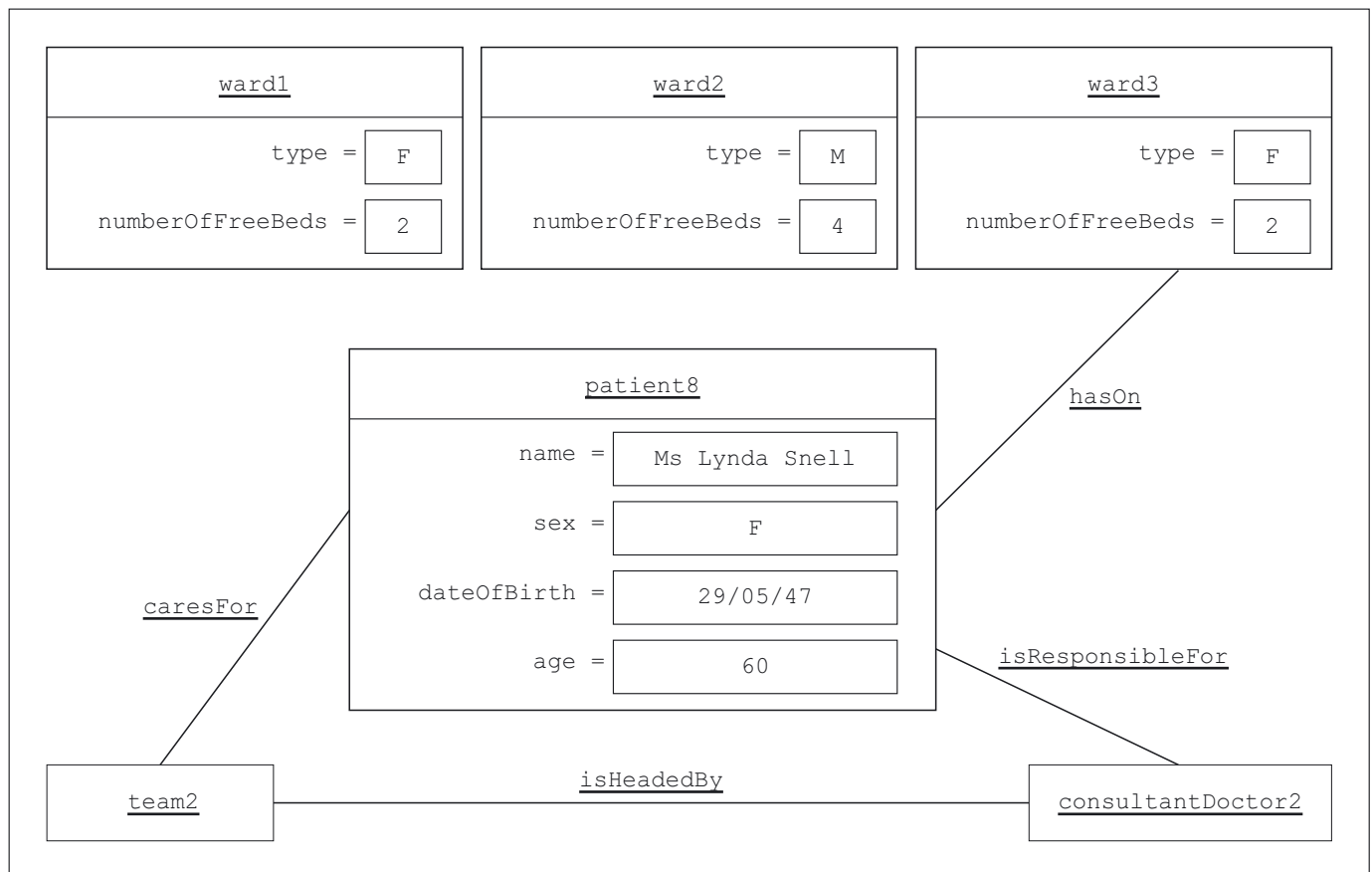


Figure 5 Object diagram for the scenario after *Admit Patient* (a suitable ward exists)

Walk-through for the above scenario.

Given: name Ms Lynda Snell, sex F, date of birth 29/05/47, and team2, a Team object.

- 1a Access the collection of Ward objects linked to hospital, which in this case is {ward1, ward2, ward3}.

For each of these Ward objects:

- 1b find out its type;
- 1c if F, get its number of free beds.
- 1d Identify ward3, the female Ward object with the greatest number of free beds.
- 2 Create patient8, a new Patient object with the supplied attribute values and age according to the supplied date.
- 3a patient8 records a reference to ward3.
- 3b ward3 records a reference to patient8, and reduces numberOfFreeBeds by 1.
- 4a patient8 records a reference to team2.
- 4b team2 records a reference to patient8.
- 7 Return ward3.

Note that in an earlier walk-through, the missing steps, 5 and 6, were intended to implement an isResponsibleFor link. However, since it was agreed in Unit 7, Subsection 4.2, that the isResponsibleFor association would be derived, these steps are no longer necessary.

Sequence diagram for the above scenario:

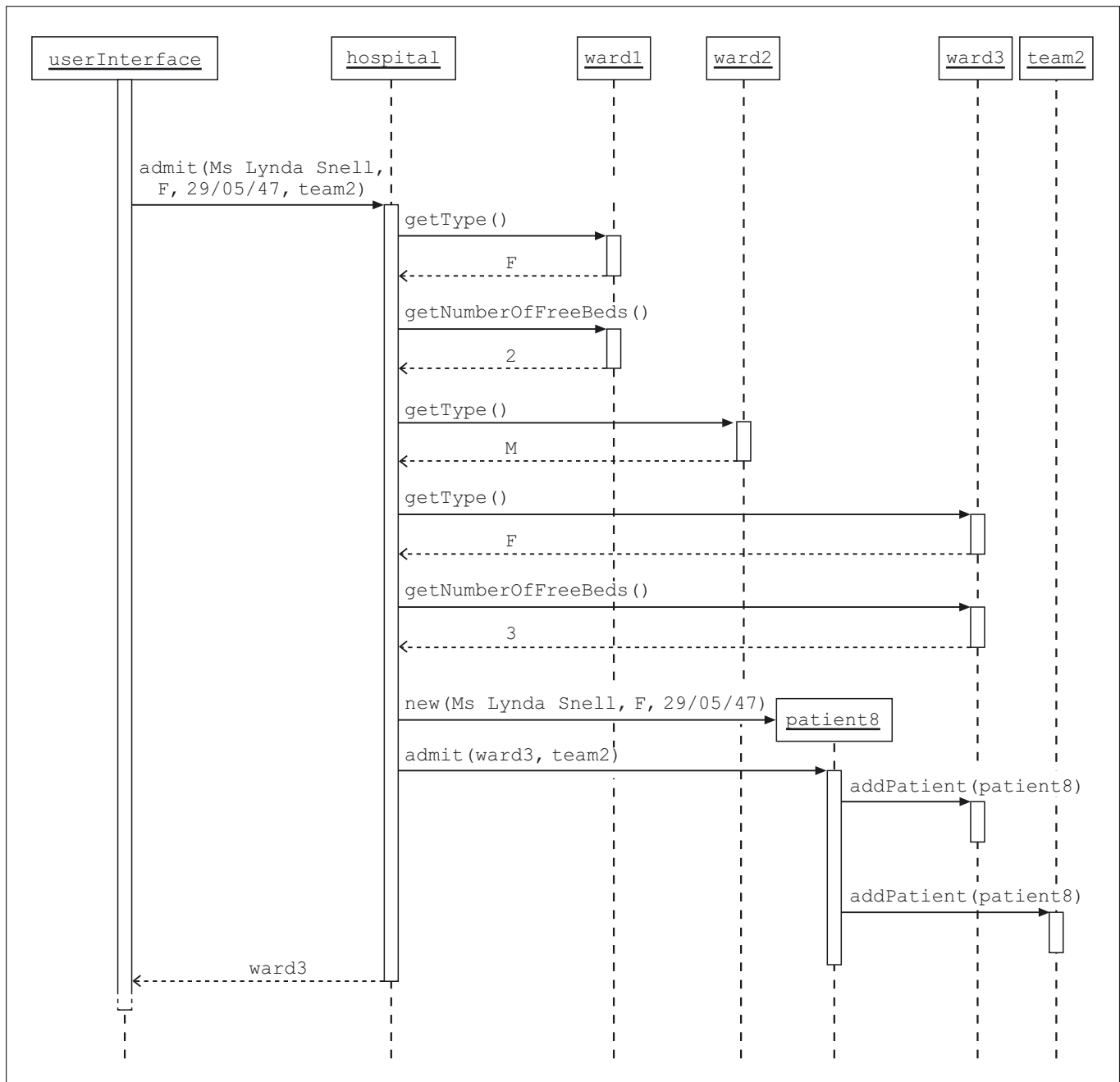


Figure 6 Sequence diagram for *Admit Patient* (a suitable ward exists)

Object diagram for an alternative scenario in which there is no suitable ward:

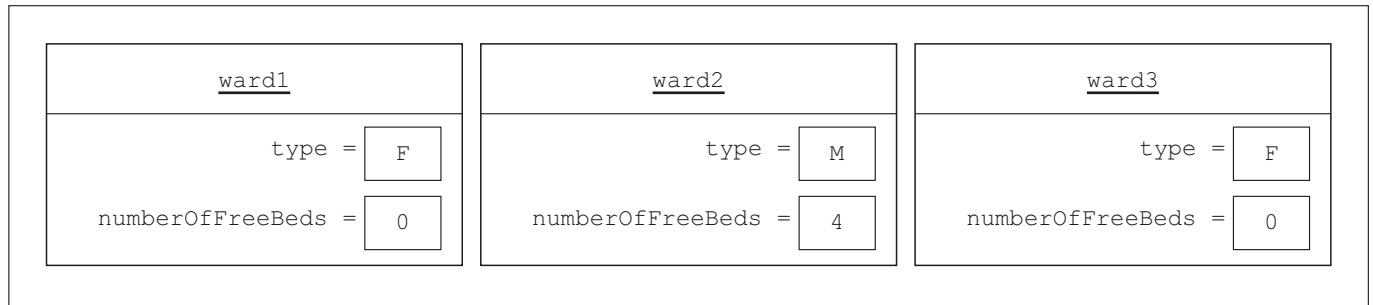


Figure 7 Object diagram for the alternative scenario before *Admit Patient* (all of the female wards are full)

Walk-through for this alternative scenario.

Given: name Ms Lynda Snell, sex F, date of birth 29/05/47, and team2, a Team object.

- 1a Access the collection of Ward objects linked to hospital, which in this case is {ward1, ward2, ward3}.

For each of these Ward objects:

- 1b find out its type;
1c if F, get its number of free beds.

As there is no Ward object with a free female bed:

- 1d return null.

Sequence diagram for this alternative scenario:

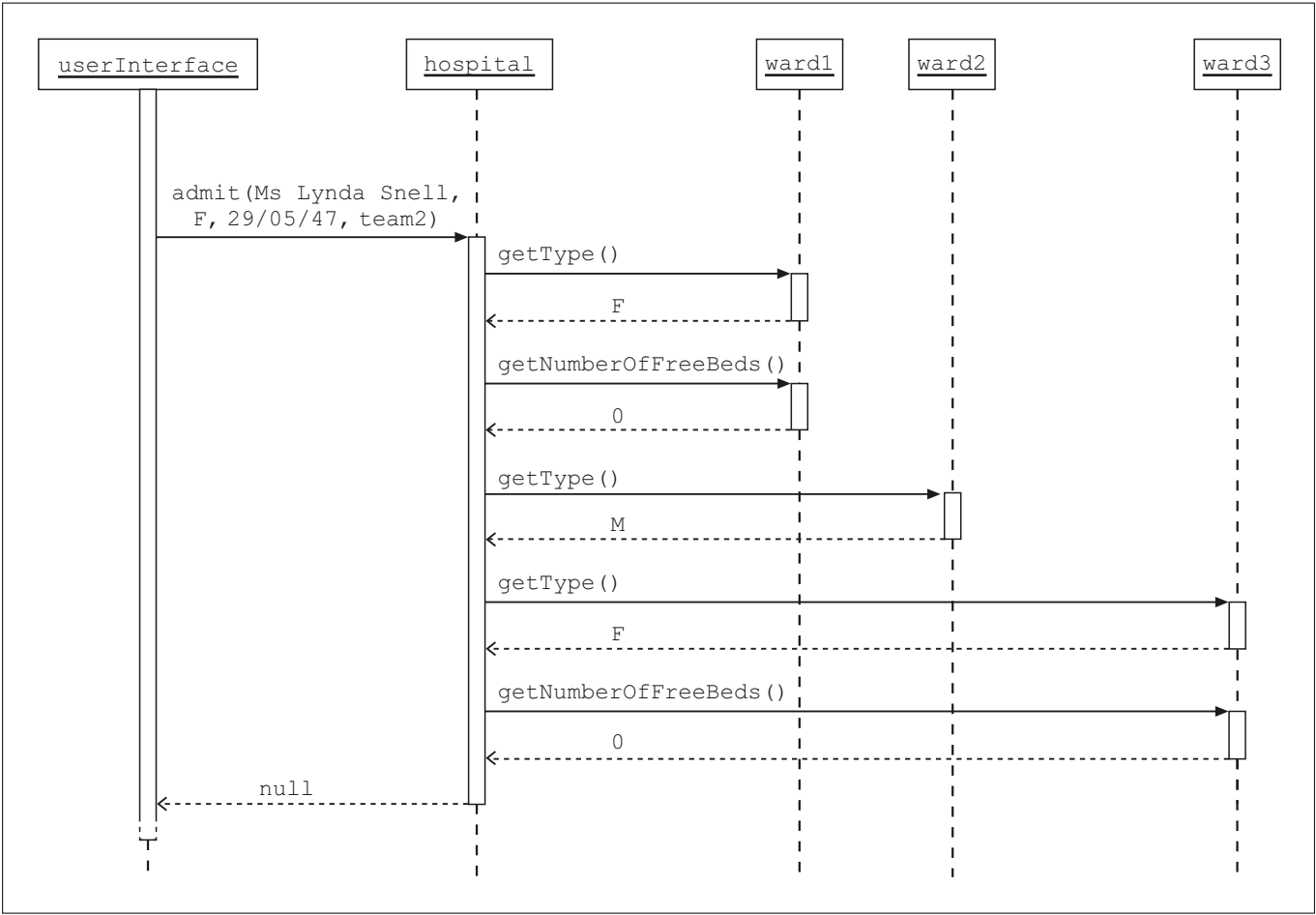


Figure 8 Sequence diagram for *Admit Patient* (alternative scenario)

B: Treat Patient

Coordinating method:

```
void recordTreatment(Patient aPatient, Doctor aDoctor)
```

Object diagrams for a typical (valid) scenario:

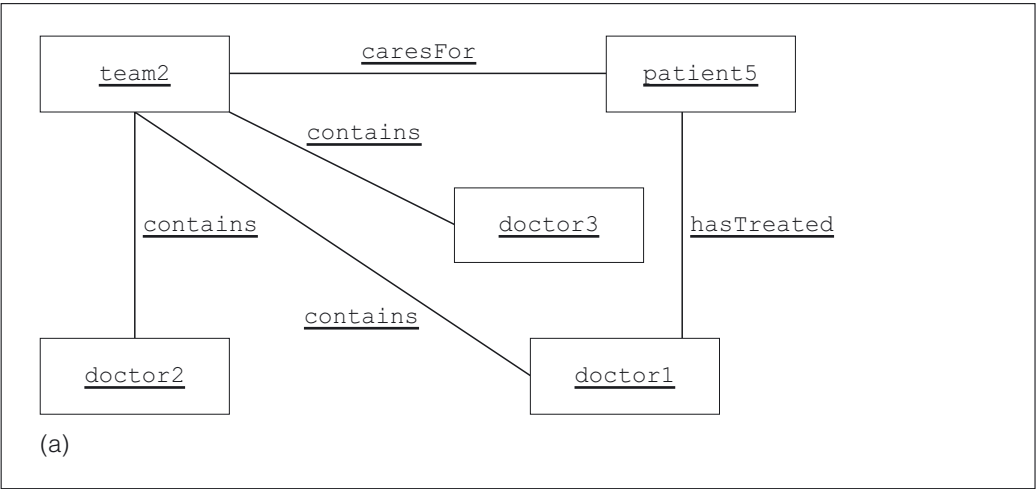


Figure 9 (a) Object diagram for the valid scenario before *Treat Patient*

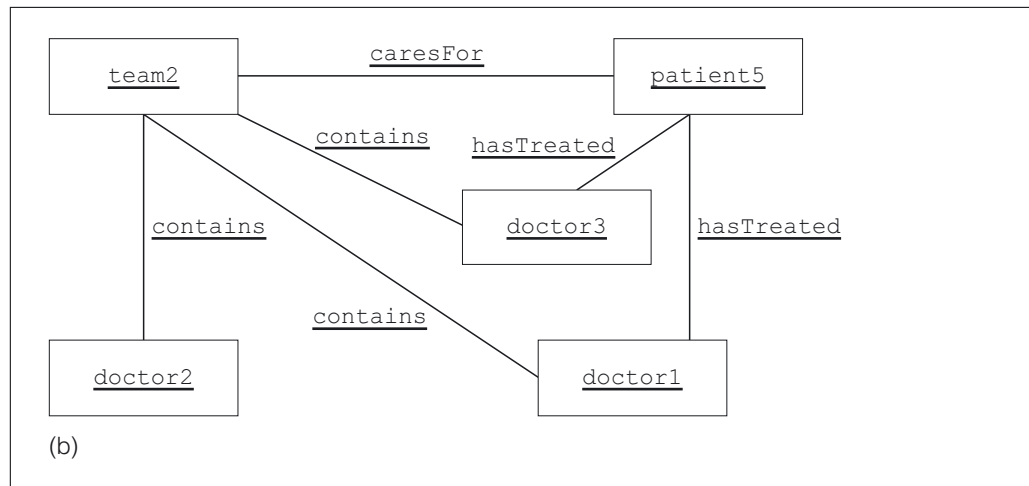


Figure 9 (b) Object diagram for the valid scenario after *Treat Patient*: a *hasTreated* link exists between *patient5* and *doctor3*.

Walk-through for this valid scenario.

Given: the Patient object *patient5*, and the Doctor object *doctor3*.

- 1 Access the Team object linked to *patient5*, which in this case is *team2*.
- 2 Access the collection of Doctor objects linked to *team2*, which in this case is {*doctor1*, *doctor2*, *doctor3*}.
- 3 Check whether *doctor3* is in {*doctor1*, *doctor2*, *doctor3*}.

As the check succeeds:

- 4 *patient5* records a reference to *doctor3*.

Sequence diagram for this valid scenario:

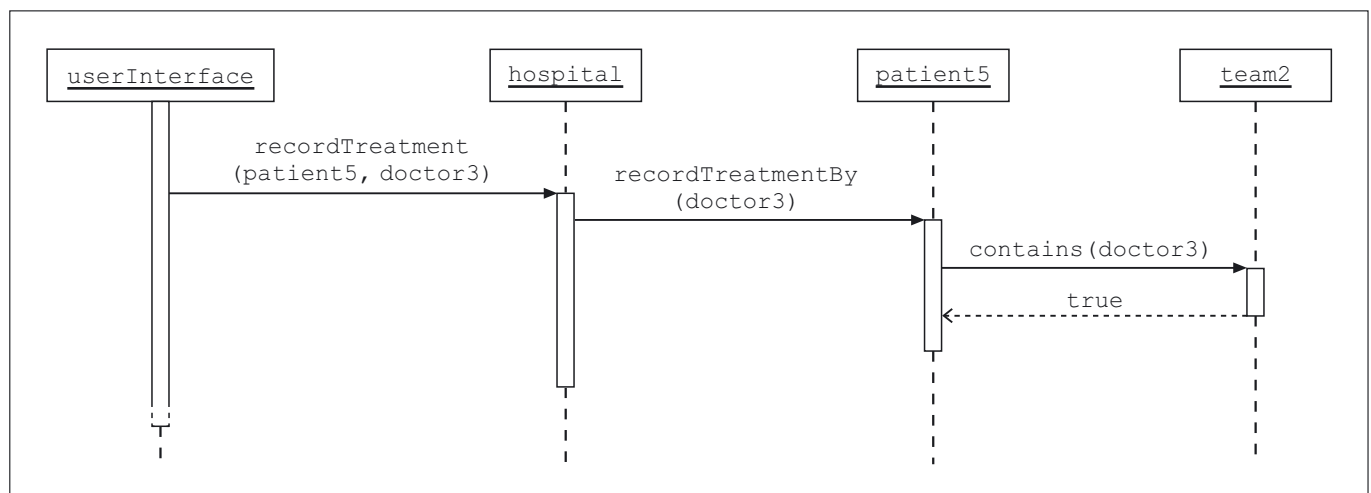


Figure 10 Sequence diagram for *Treat Patient* (valid scenario)

Object diagram for an invalid scenario for *Treat Patient*:

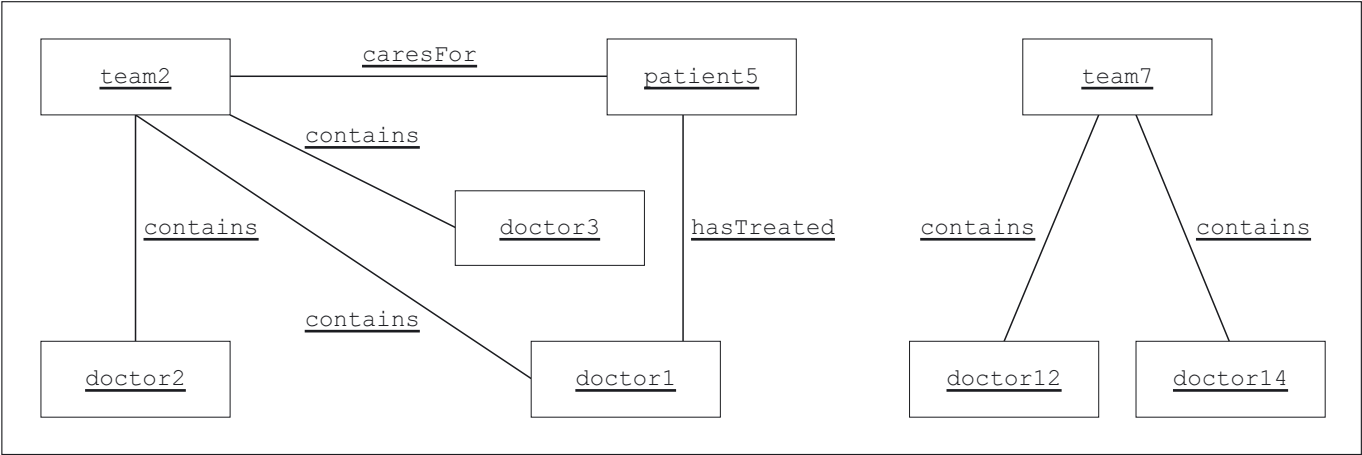


Figure 11 Object diagram for the invalid scenario for *Treat Patient*: `patient5` should not be treated by `doctor12`

Walk-through for this invalid scenario.

Given: the Patient object `patient5`, and the Doctor object `doctor12`.

- 1 Access the Team object linked to `patient5`, which in this case is `team2`.
- 2 Access the collection of Doctor objects linked to `team2`, which in this case is {`doctor1`, `doctor2`, `doctor3`}.
- 3 Check whether `doctor12` is in {`doctor1`, `doctor2`, `doctor3`}.

As the check fails:

- 4 throw an exception.

Sequence diagram for this invalid scenario:

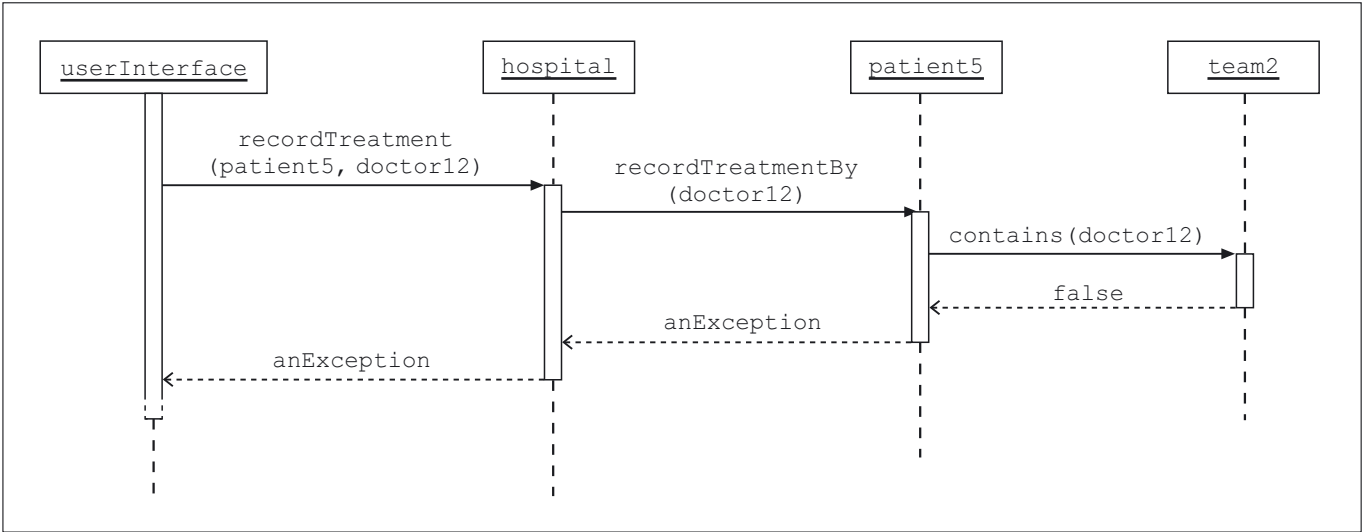


Figure 12 Sequence diagram for *Treat Patient* (invalid scenario)

C: Discharge Patient – omitted

D: Transfer Patient – omitted

E: List Ward's Patients

Coordinating method:

```
Collection<Patient> getPatients(Ward aWard)
```

Object diagram for a typical scenario:

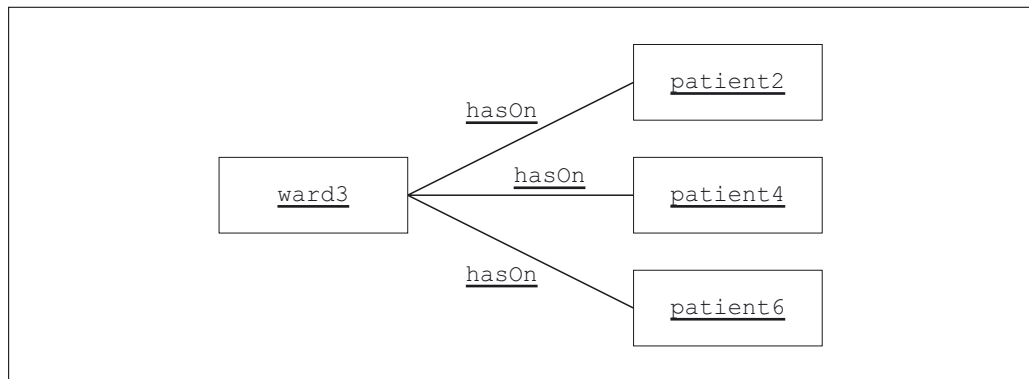


Figure 13 Object diagram for the *List Ward's Patients* scenario

Walk-through for this scenario.

Given: the Ward object, ward3.

- 1 Access the collection of Patient objects linked to ward3, which in this case is {patient2, patient4, patient6}.
- 2 Return {patient2, patient4, patient6}.

Sequence diagram for this scenario:

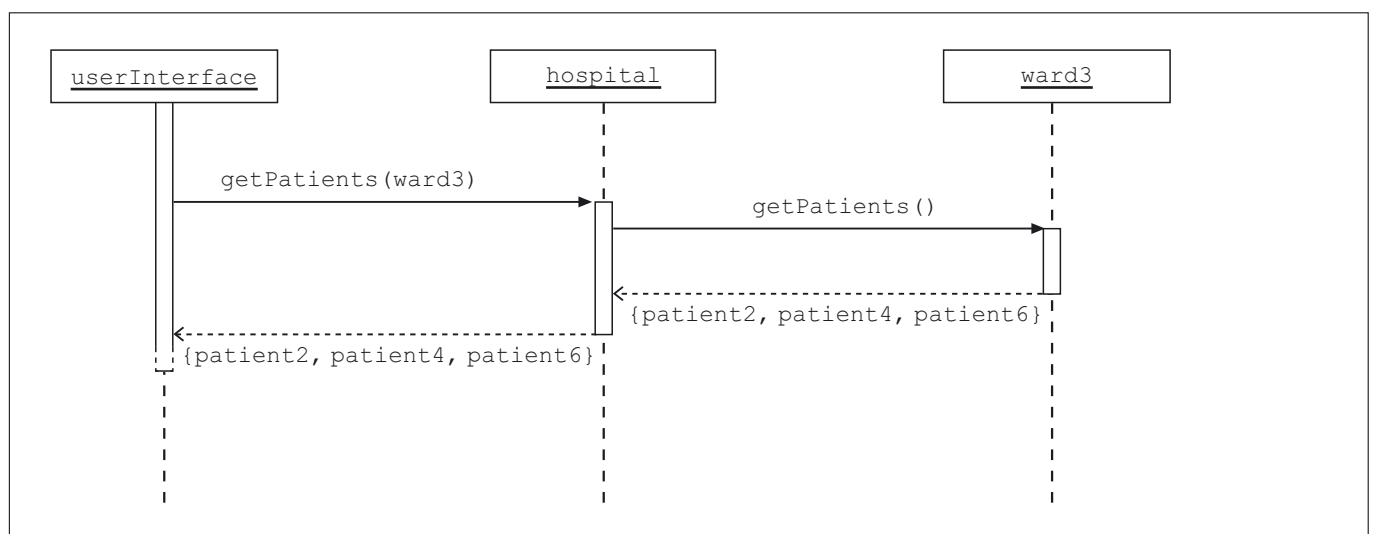


Figure 14 Sequence diagram for *List Ward's Patients*

F: List Team’s Patients

Coordinating method:

```
Map<Patient, Ward> getPatientsAndWards(Team aTeam)
```

Object diagram for a typical scenario:

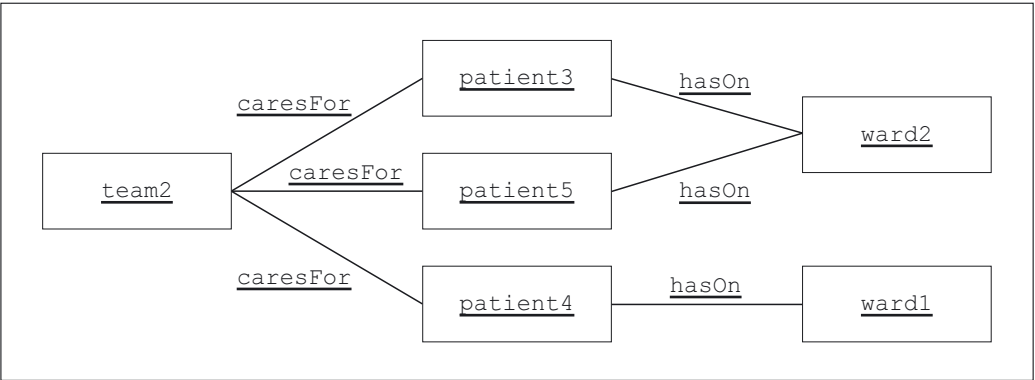


Figure 15 Object diagram for the scenario for *List Team's Patients*

Walk-through for this scenario.

- Given: the Team object team2.
- 1 Access the collection of Patient objects linked to team2, which in this case is {patient3, patient4, patient5}.
 - 2 Create a new empty collection, results.
- For each Patient object, aPatient, accessed in step 1:
- 3 access aWard, the Ward object linked to aPatient;
 - 4 add (aPatient, aWard) to results.
- 5 Return results.

Sequence diagram for this scenario:

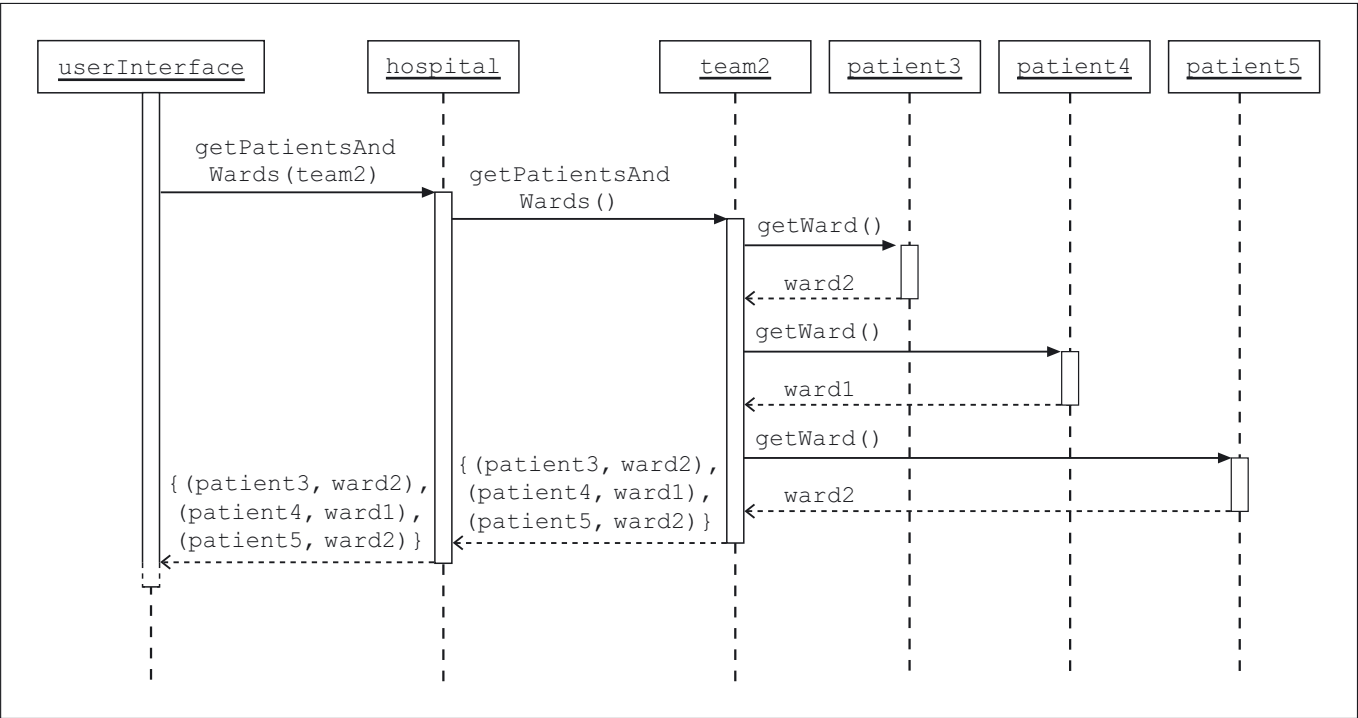


Figure 16 Sequence diagram for *List Team's Patients*

G: List Patient's Treatment

Three coordinating methods:

```
ConsultantDoctor getConsultantDoctor(Patient aPatient)
Team getTeam(Patient aPatient)
Collection<Doctor> getDoctors(Patient aPatient)
```

Object diagram for a typical scenario:

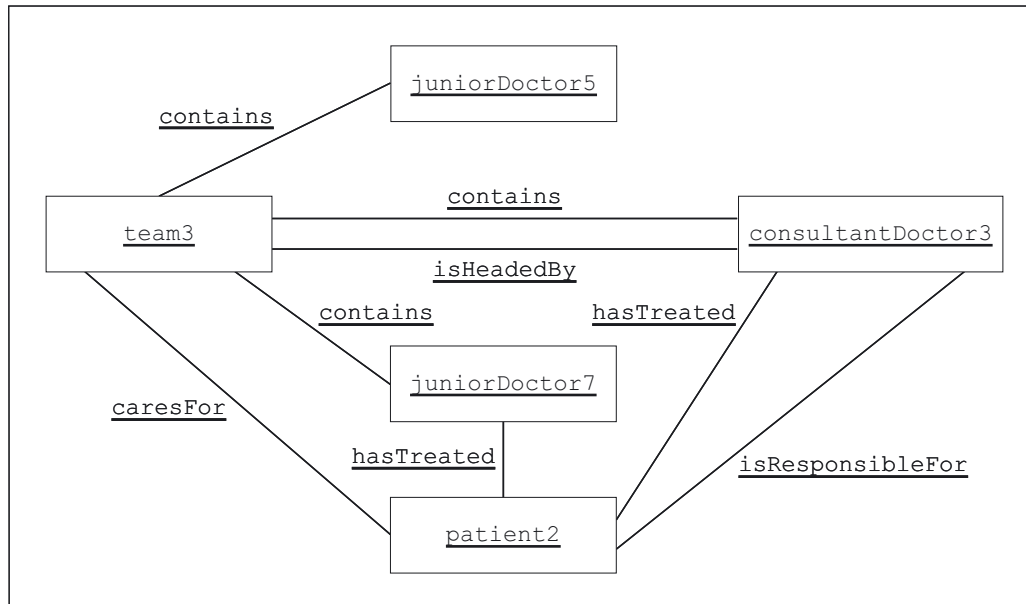


Figure 17 Object diagram for the scenario for *List Patient's Treatment*

Walk-through for this scenario for `getConsultantDoctor(aPatient)`.

Given: the Patient object `patient2`.

- 1 Access the Team object linked to `patient2`, which in this case is `team3`.
- 2 Access the ConsultantDoctor object linked to `team3`, which in this case is `consultantDoctor3`.
- 3 Return `consultantDoctor3`.

Sequence diagram for this scenario for `getConsultantDoctor(aPatient)`:

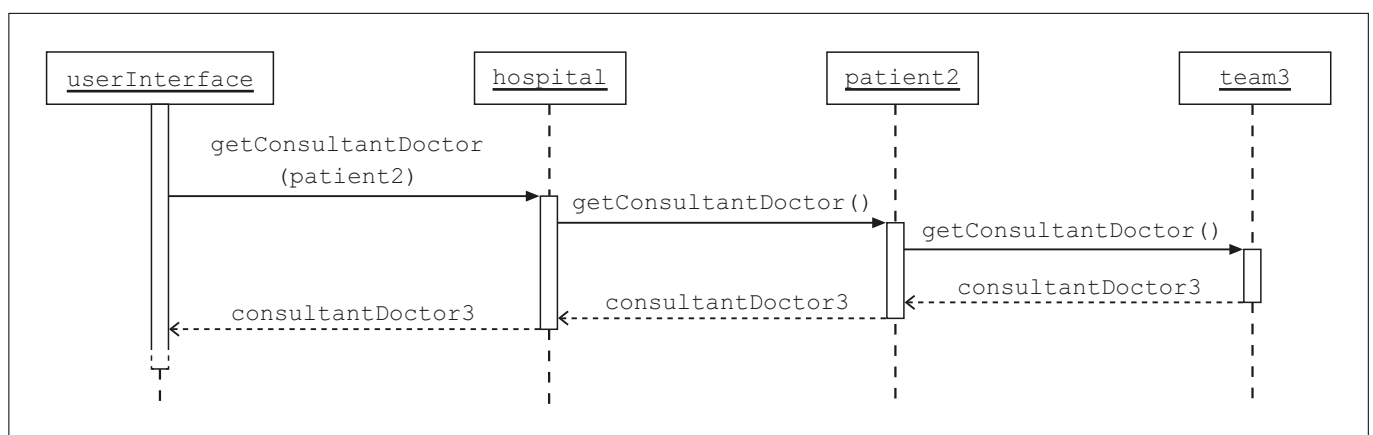


Figure 18 Sequence diagram for *List Patient's Treatment* coordinating method: `getConsultantDoctor(aPatient)`

Walk-through for this scenario for `getTeam(aPatient)`.

- Given: the Patient object `patient2`.
- 1 Access the Team object linked to `patient2`, which in this case is `team3`.
 - 2 Return `team3`.

Sequence diagram for this scenario for `getTeam(aPatient)`:

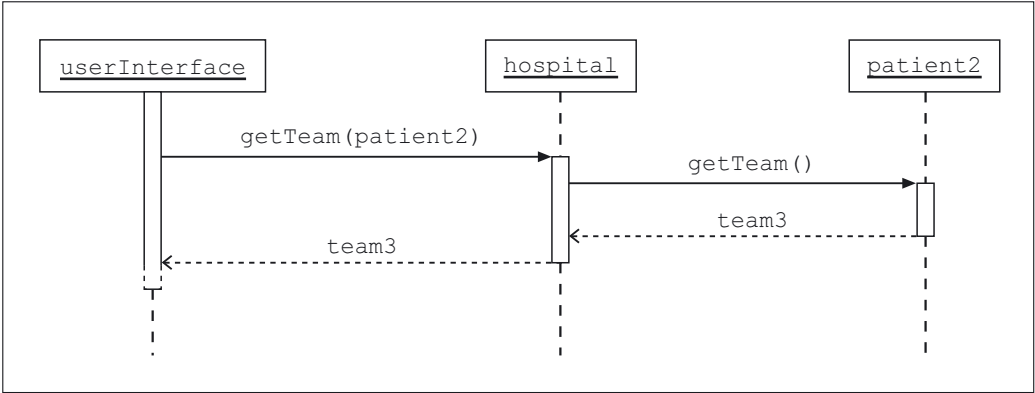


Figure 19 Sequence diagram for *List Patient's Treatment* coordinating method: `getTeam(aPatient)`

Walk-through for this scenario for `getDoctors(aPatient)`.

- Given: the Patient object `patient2`.
- 1 Access the collection of Doctor objects linked to `patient2`, which in this case is `{consultantDoctor3, juniorDoctor7}`.
 - 2 Return `{consultantDoctor3, juniorDoctor7}`.

Sequence diagram for above scenario for `getDoctors(aPatient)`:

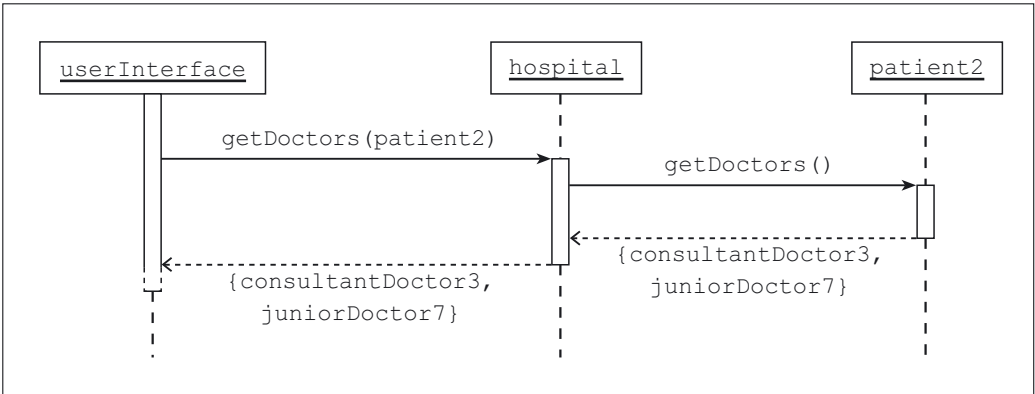


Figure 20 Sequence diagram for *List Patient's Treatment* coordinating method: `getDoctors(aPatient)`

6 Shopping System requirements document

The requirements document for the Shopping System

System domain

The Open Shopping Company sells computing products on its website. The company has a number of registered customers. A registered customer may visit the website to view product details, and to make and place an order. A customer must be authenticated by the system before such an online session can commence. A customer may only be involved in a single session at any one time.

Once a customer has started an online session, and opts to create an order, an empty order is created, and the customer can add one item of an available product to their order at a time. As many items as required may be added to an order. Multiple items of the same product must be added singly.

If a selected product is out of stock, and the company has a suitable alternative in stock, the system will suggest this alternative to the customer, who may select or reject it. Alternative products are paired, so that if product A is considered by the company to be a suitable alternative to product B, then product B is considered to be a suitable alternative to product A. Each product may only have a single alternative and not all items have alternatives.

The customer may submit or discard an order at any time during the session. It is not possible to add any further items to an order once it has been submitted. A customer may submit more than one order during a single session, but may only have one unsubmitted order at a time.

Use cases

The system will allow a customer to do the following before a session.

- A Record Customer Registration.** The customer provides the system with the customer's name, address and chosen password. The system provides a unique customer number and records the customer's details. The customer number is displayed.
- B Start Session.** The customer provides the system with a customer number and a password. If the entered password matches the password recorded for the customer with that number, the system records a current session for that customer. If the customer is not authenticated in this way, the system displays an appropriate message.

The system will allow a customer with a current session to do the following.

- C Terminate Session.** The system records that the customer no longer has a current session. If the customer has an unsubmitted order this is discarded (see below). All information about the session is removed from the system.
- D List Submitted Orders.** For each order submitted by the customer, the system displays the product code, description and price (at time of ordering) of each item in the order, and the order number, date submitted and total cost of the order.

- E List Products.** For each product the system displays the product code, description and price.
- F Create Order.** The system creates an empty order for the customer and generates a unique order number.
- G Add Item.** The customer identifies a product. If the product is in stock, the system records that an item of the product has been added to the customer's current order. If the product is out of stock, the system displays an appropriate message. If there is an alternative to the product, and there are items of this alternative in stock, then the alternative is suggested to the customer. If the customer accepts the alternative the system records that an item of the alternative product has been added to the customer's current order. The system decrements the stock level of the product added to the order.
- H Display Order.** The system displays the product code, description and price (at time of ordering) of each item in the order, and the date, order number and total cost of the order.
- I Submit Order.** If the order contains at least one item, the system records that the order has been submitted.
- J Discard Order.** The system increments the stock level of each product with items in that order. All information about the order is removed from the system.

The system will allow a staff member to do the following.

- K List Registered Customers.** For each registered customer the system displays the name, address and customer number.
- L List Customers With Current Session.** For each customer with a current session the system displays the time the session started, the customer's name and customer number, and indicates whether that customer has an unsubmitted order.
- M Record New Product.** The staff member provides the system with the product's unique product code, description, stock level and price. If an alternative product is available, the staff member identifies it. Such an alternative product should not already be an alternative for another product. The system records the new product and, if relevant, records that it and the identified alternative are alternatives.
- N List Products.** For each product the system displays the product code, description, price and stock level.
- O Record Product Change.** The staff member identifies a product and either provides the system with a new stock level or price, or identifies an alternative product (which should not already be an alternative for another product). The system records the changed product details for one or both products. Items of this product that have already been added to a customer's order are not affected by an updated price.
- P List Submitted Orders.** The staff member identifies a customer. The system lists that customer's submitted orders as for use case D.

Acceptance tests

- A** Test that a new customer is registered correctly.
- B** Test that a session is initiated correctly for a legitimate customer.
Test that an attempt to initiate a session by a non-legitimate customer results in an appropriate message.
- C** Test that a session is terminated correctly if the customer has no unsubmitted order.

- Test that a session is terminated correctly if the customer has an unsubmitted order.
- D** Test that a list of submitted orders is produced correctly when there are no orders submitted by the customer.
Test that a list of submitted orders is produced correctly when there are orders submitted by the customer.
- E** Test that a list of products is produced correctly when there are no products recorded.
Test that a list of products is produced correctly when there are products recorded.
- F** Test that an empty order is produced correctly.
- G** Test that a product item is added correctly to an order if it is available.
Test that a product item is added correctly if the original product item is not available, but an alternative is available and is accepted.
Test that a product item is not added if the original product item is not available and there is no alternative available.
Test that a product item is not added if the original product item is not available, an alternative is available and is not accepted.
- H** Test that an order is displayed correctly when there are no items in the order.
Test that an order is displayed correctly when there are items in the order.
- I** Test that a non-empty order is correctly recorded as submitted.
- J** Test that an order is correctly discarded when there are no items in the order.
Test that an order is correctly discarded when there are items in the order.
- K** Test that a list of registered customers is produced correctly when there are no customers recorded.
Test that a list of registered customers is produced correctly when there are customers recorded.
- L** Test that a list of customers with current sessions is produced correctly when there are no customers with current sessions.
Test that a list of customers with current sessions is produced correctly when there are customers with current sessions, none of whom have an unsubmitted order.
Test that a list of customers with current sessions is produced correctly when there are customers with current sessions, some of whom have an unsubmitted order.
- M** Test that a new product with no identified alternative is recorded correctly.
Test that a new product with an identified alternative is recorded correctly and that the product and the alternative are recorded as alternatives.
- N** Test that a list of products is produced correctly when there are no products recorded.
Test that a list of products is produced correctly when there are products recorded.
- O** Test that, when changing the alternative product is not part of the change, a product change is recorded correctly.
Test that, when changing the alternative product is part of the change, a product change is recorded correctly, including the fact that the new product and the alternative are recorded as alternatives.

- P** Given a customer:
- ▶ test that a list of submitted orders is produced correctly when there are no orders submitted by that customer;
 - ▶ test that a list of submitted orders is produced correctly when there are some orders submitted by that customer.

7 Shopping System initial structural model

Class diagram

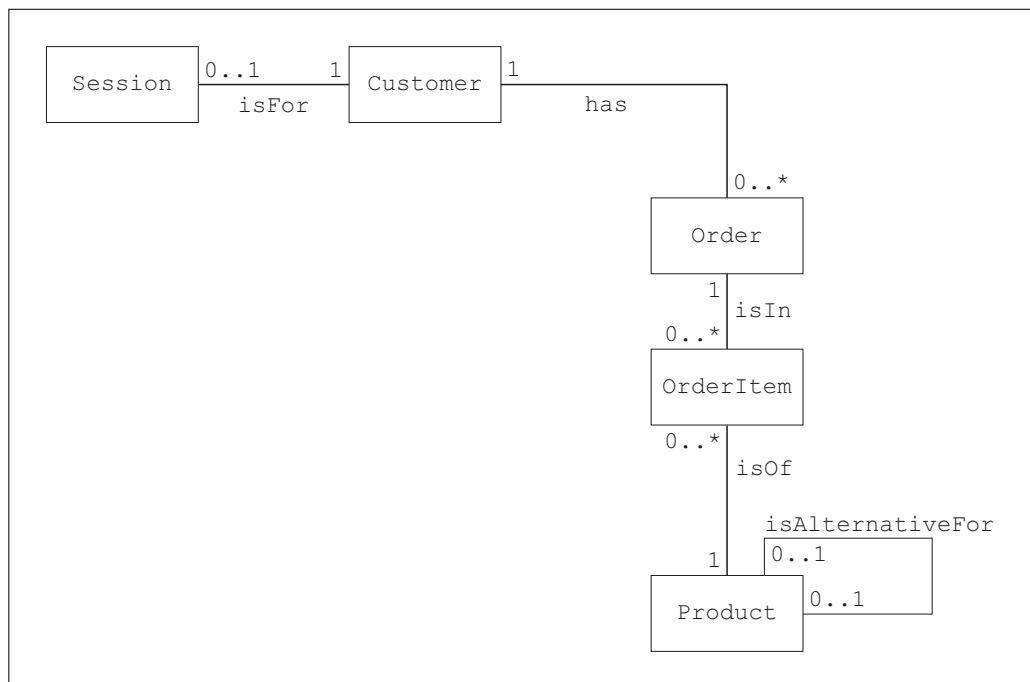


Figure 21 Class diagram for the Shopping System initial structural model

Class descriptions

Class Session A customer session on the company website

Attributes

startTime The time the session started

Class Customer A customer of the company

Attributes

customerNumber The unique number of the customer

name The name of the customer

address The address of the customer

password The password of the customer

Class	<code>Order</code>	A customer's order
Attributes		
	<code>orderNumber</code>	The unique number of the order
	<code>/totalCost</code>	The total cost of the order
	<code>date</code>	The date of the order
	<code>status</code>	The status (submitted or unsubmitted) of the order

Class	<code>OrderItem</code>	An item in a customer's order
Attributes		
	<code>price</code>	The price of the item at the time of selection

Class	<code>Product</code>	A product stocked by the company
Attributes		
	<code>code</code>	The unique code of the product
	<code>description</code>	The description of the product
	<code>price</code>	The current price of the product
	<code>stockLevel</code>	The current stock level of the product

Invariants

- 1 A `Customer` object may be linked to no more than one `Order` object whose `status` attribute has the value `unsubmitted`.
- 2 If a `Customer` object is linked to an `Order` object whose `status` attribute has the value `unsubmitted`, then that `Customer` object is also linked to a `Session` object.
- 3 For each `Order` object, its `totalCost` attribute value is equal to the sum of the `price` attribute values of all the `OrderItem` objects to which the `Order` object is linked.
- 4 Each `Order` object whose `status` attribute has the value `submitted` is linked to at least one `OrderItem` object.
- 5 No `Product` object may be linked to itself.

8

DVD Library System requirements document

The requirements document for the DVD Library System

(Acceptance tests omitted.)

System domain

The DVD Library holds DVDs of a number of films. It may have any number of copies of each film. The system should, for each film, hold the film's title; no two films have the same title.

A library member can borrow any DVD of a film that is not currently on loan. However, no member can have more than six DVDs on loan at once. A borrowed DVD must be returned within three days. When a library member returns a DVD it immediately becomes available for loan to other members.

Each DVD has a unique identifying number.

Each member of the library has a unique membership number; the system also holds each member's name.

Use cases

The DVD Library System will provide support to the library by enabling the library staff to do the following:

- A Get Number Available.** The staff member identifies the film. The system displays the number of copies available.
- B List Films.** The staff member identifies the member. The system displays a list of the film titles and return dates for the DVDs that the member has on loan.
- C Borrow DVD.** The staff member identifies the DVD and the member who is borrowing it. The system records the fact that this copy of the film is no longer available for loan, the fact that it is on loan to the member identified, and the date on which it was borrowed.
- D Return DVD.** The staff member identifies the DVD. The system records the fact that this copy of the film is now available for loan and that it is no longer on loan to the member who returned it.

9

DVD Library System initial structural model

Class diagram

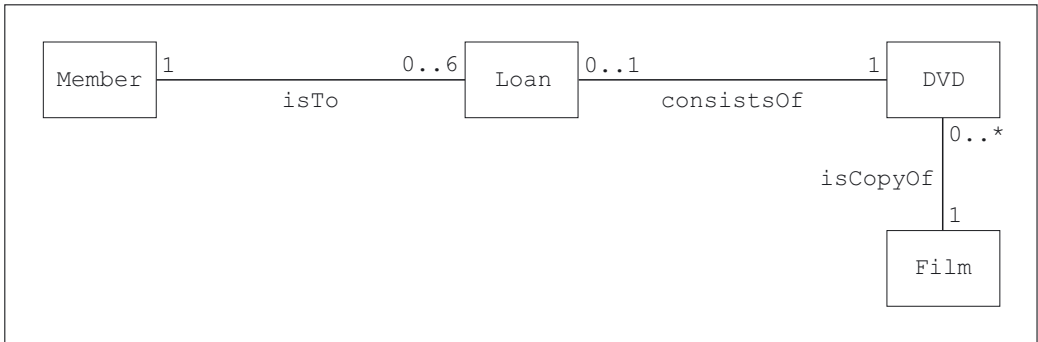


Figure 22 Class diagram for the DVD Library System initial structural model.

Class descriptions

Class	DVD	A DVD of a film
	Attributes	
	number	The unique identifying number of the DVD
Class	Film	A film
	Attributes	
	title	The unique title of the film
	/numberAvailable	The number of copies of the film available for loan
Class	Loan	The loan of a DVD
	Attributes	
	issueDate	The date of issue of the loan
	/returnDate	The date by which the loaned DVD must be returned.
Class	Member	A library member
	Attributes	
	number	The unique membership number of the member
	name	The name of the member

Invariants

- 1 For each Loan object, the difference between the values of its issueDate and returnDate attributes is 3 days.
- 2 For any Film object, the value of its numberAvailable attribute is equal to the number of DVD objects it is linked to which are not linked to a Loan object.